Projektanforderungen OOPGames

Programmieren 3 · Wintersemester 2026/27 · Josef Griesbauer

Dieses Dokument legt verbindlich fest, wie die Projektarbeit der Studierendengruppen am Framework `OOPGames` (WPF, C#, .NET 6) abläuft. Die Arbeit besteht aus zwei Anforderungen:

1. **Anforderung 1 – Vier gewinnt (gruppenübergreifend kompatibel):** Alle Gruppen implementieren „Vier gewinnt" so, dass Painter, Rules und Player *aller* Gruppen beliebig miteinander kombiniert werden können.
2. **Anforderung 2 – Eigenes komplexes Spiel:** Jede Gruppe entwickelt mit Hilfe von KI-Agenten ein eigenes, komplexes bzw. innovatives Spiel, das im vorgegebenen Framework läuft.

Die Formulierungen MUSS / DARF NICHT kennzeichnen harte Vorgaben; „soll" kennzeichnet dringende Empfehlungen.

1 Das Framework im Überblick

Das Framework trennt jedes Spiel in drei austauschbare Rollen, die im Hauptfenster unabhängig voneinander ausgewählt werden:

Rolle	Basis-Interfaces	Aufgabe
Painter	`IPaintGame`, `IPaintGame2`	Zeichnet ein `IGameField` auf das Canvas (nach jedem Zug bzw. alle 40 ms bei `IPaintGame2`).
Rules	`IGameRules`, `IGameRules2`, `IGameField`	Hält das Spielfeld, führt Züge aus, prüft Gewinner und ob noch Züge möglich sind; `IGameRules2` erlaubt Echtzeit-Spiele über `TickGameCall()`.
Player	`IHumanGamePlayer`, `IComputerGamePlayer`	Erzeugt Züge (`IPlayMove`) – aus Maus-/Tastatureingaben (`IClickSelection`, `IKeySelection`) oder als Computergegner.

Die Kompatibilität wird zur Laufzeit über `IGameField.CanBePaintedBy(painter)` und `IGamePlayer.CanBeRuledBy(rules)` geprüft. Das Referenzbeispiel `X_Griesbauer` (TicTacToe) zeigt das Muster: spielspezifische Interfaces (`IX_TicTacToeField`, `IX_TicTacToeRules`, `IX_PaintTicTacToe`, …) erweitern die Basis-Interfaces, und die Prüfungen erfolgen ausschließlich gegen diese *Interfaces*, nie gegen konkrete Klassen. Dieses Muster ist auch für Ihre Implementierungen verbindlich.

2 Generelle Anforderungen (gelten für beide Teile)

2.1 Verzeichnisstruktur und Namenskonventionen

- Jede Gruppe arbeitet ausschließlich in ihren eigenen Verzeichnissen – analog zu `X_Griesbauer`:

```
OOPGames/OOPGames/Classes/<Gruppe>/       ← Klassen der Gruppe (z. B. Classes/A1_Gruppe/)
OOPGames/OOPGames/Interfaces/<Gruppe>/    ← gruppeneigene Interfaces
```

Die Gruppe legt beide Verzeichnisse selbst an; `<Gruppe>` ist das in der Übung vergebene Gruppenkürzel (z. B. `A1_Gruppe`). Im Folgenden steht `A1` stellvertretend für das eigene Kürzel.

- Dateinamen folgen dem Schema `Vorname_Nachname_<Inhalt>.cs` (Interfaces: `Vorname_Nachname_I_<Inhalt>.cs`). Die Datei gehört damit sichtbar dem Studierenden, der sie hauptverantwortlich erstellt.
- Alle Klassen- und Interfacenamen tragen das Gruppenpräfix (z. B. `A1_VierGewinntRules`, `IA1_…`), damit es keine Namenskollisionen im gemeinsamen Namespace `OOPGames` gibt.
- Die `Name`-Properties von Painter, Rules und Player sind eindeutig und enthalten Gruppe bzw. Namen (z. B. `"A1 Mustermann VierGewinnt Painter"`), damit sie in den Auswahllisten unterscheidbar sind.
- Die einzige erlaubte Änderung außerhalb der eigenen Verzeichnisse ist das **Registrieren** der eigenen Klassen im markierten Block `//REGISTER YOUR CLASSES HERE` in `MainWindow.xaml.cs` (siehe Abschnitt 2.4).

2.2 Git: Jedes Teammitglied committet und pusht selbst

- Jedes Teammitglied MUSS seine Arbeit mit dem **eigenen GitHub-Account** selbst committen und pushen. Die Commit-Historie dient als Nachweis der Aufgabenverteilung im Team; Sammel-Commits einer Person für das ganze Team sind nicht zulässig.
- Regelmäßig und in kleinen Schritten committen; aussagekräftige Commit-Messages (was und warum), z. B. `A1: Gewinnprüfung Diagonalen in VierGewinntRules`.
- Vor jedem Push: `git pull` und sicherstellen, dass das Gesamtprojekt baut (`dotnet build`). Ein Push, der den Build für alle Gruppen bricht, ist umgehend zu beheben.
- `git push --force`, das Umschreiben fremder Historie und das Ändern/Löschen fremder Dateien sind untersagt.
- Keine Build-Artefakte (`bin/`, `obj/`) oder persönlichen IDE-Einstellungen committen.

2.3 Dokumentation der agentischen Entwicklung: Prompts und Pläne

Da die Implementierung mit KI-Agenten erfolgt, muss die Entstehung nachvollziehbar sein. Als **Hauptverzeichnis der Gruppe** gilt `OOPGames/OOPGames/Classes/<Gruppe>/`.

- **Prompts:** Für jeden Studierenden existiert im Hauptverzeichnis der Gruppe eine eigene Markdown-Datei `Vorname_Nachname_Prompts.md`, in der jeder an einen Agenten gerichtete Prompt hinterlegt ist – vollständig im Wortlaut, chronologisch, mit Datum, verwendetem Werkzeug/Modell und einer kurzen Notiz zum Ergebnis (übernommen / verworfen / nachgebessert).
- **Pläne:** Alle Pläne – sowohl die vom Agenten erzeugten (z. B. aus einem Plan-Modus) als auch eigene Planungen des Teams – werden im Unterordner `Plans/` des Hauptverzeichnisses als Markdown abgelegt (Schema: `JJJJ-MM-TT_Vorname_Nachname_<Thema>.md`).
- Prompt- und Plandateien werden **zusammen mit dem zugehörigen Code** committet, sodass sich Prompt, Plan und Ergebnis zuordnen lassen.

```
Classes/A1_Gruppe/
├── Max_Mustermann_Prompts.md
├── Erika_Musterfrau_Prompts.md
├── Plans/
│   ├── 2026-11-05_Max_Mustermann_VierGewinntRules.md
│   └── 2026-11-12_Erika_Musterfrau_Spielkonzept.md
├── Scrum/            (siehe Abschnitt 4.4)
├── Max_Mustermann_VierGewinntRules.cs
└── Erika_Musterfrau_VierGewinntPainter.cs
```

Vorschlag für das Format eines Prompt-Eintrags:

```
## 2026-11-05 – Claude Code (Modell: …)
**Prompt:**
> Implementiere die Klasse A1_VierGewinntRules gemäß den Interfaces in
> Interfaces/C4_Gemeinsam/ … Ändere keine Dateien außerhalb von Classes/A1_Gruppe/.

**Ergebnis:** übernommen, Gewinnprüfung für Diagonalen manuell korrigiert.
```

2.4 Framework-Dateien sind tabu

> **Harte Vorgabe: Das Framework DARF NICHT eigenmächtig verändert werden – auch nicht durch einen Agenten.**
>
> Zum Framework gehören insbesondere: alle Dateien direkt in `Interfaces/` (`IGamePlayer.cs`, `IGameRules.cs`, `IPaintGame.cs`, `IPlayMoves.cs`), `Classes/OOPGames.cs`, `MainWindow.xaml`, `MainWindow.xaml.cs` (außer dem Registrierungsblock), `App.*`, `OOPGames.csproj`, `.sln`, `.vscode/` sowie die Verzeichnisse `X_Griesbauer` und die Verzeichnisse anderer Gruppen.

- Ist eine Erweiterung des Frameworks aus Sicht der Gruppe unumgänglich, MUSS sie **vorab mit dem Dozenten abgeklärt** werden (kurze schriftliche Begründung: was soll geändert werden, warum reicht das bestehende Framework nicht, welche Auswirkungen hat es auf andere Gruppen). Erst nach Freigabe darf die Änderung umgesetzt werden.
- Die Gruppe ist dafür verantwortlich, dass ihr Agent diese Regel einhält. Jeder Prompt bzw. die Projektanweisung des Agenten (z. B. eine eigene Instruktionsdatei oder ein fester Prompt-Baustein) soll ausdrücklich enthalten: *„Ändere ausschließlich Dateien in Classes/<Gruppe>/ und Interfaces/<Gruppe>/. Wenn eine Änderung am Framework nötig erscheint, brich ab und beschreibe die benötigte Änderung, statt sie umzusetzen."*
- Vor jedem Commit ist mit `git status` / `git diff` zu prüfen, dass der Agent keine Dateien außerhalb der erlaubten Bereiche verändert hat. Unerlaubte Framework-Änderungen werden zurückgesetzt und gehen negativ in die Bewertung ein.

3 Anforderung 1: Vier gewinnt – kompatibel über alle Gruppen

3.1 Ziel

Jede Gruppe implementiert für „Vier gewinnt" mindestens **einen Painter, ein Rules-Objekt (mit Spielfeld), einen menschlichen Player und einen Computer-Player**. Im Hauptfenster muss jede

Kombination funktionieren, also z. B. der Painter von A1 mit den Rules von B3, Player 1 von A5 (Mensch) und Player 2 von B2 (Computer).

3.2 Gemeinsame Interfaces als Vertrag

- Grundlage der Kompatibilität ist ein **gemeinsamer Satz von Vier-gewinnt-Interfaces** in einem eigenen, gemeinsamen Verzeichnis (z. B. `Interfaces/C4_Gemeinsam/`, Präfix `IC4_`), aufgebaut nach dem Muster von `Josef_Griesbauer_ITicTacToe.cs`:
  - `IC4_Field : IGameField` – Spielfeld mit Indexer `this[int r, int c]` (0 = leer, 1/2 = Spieler) sowie Zeilen-/Spaltenanzahl (Standard 6 × 7);
  - `IC4_Move : IColumnMove` – ein Zug ist die Auswahl einer Spalte;
  - `IC4_Rules : IGameRules` – Zugriff auf das `IC4_Field` und Ausführen eines `IC4_Move`;
  - `IC4_Painter : IPaintGame` – Zeichnen eines `IC4_Field`;
  - `IC4_HumanPlayer : IHumanGamePlayer` und `IC4_ComputerPlayer : IComputerGamePlayer`.
- Der genaue Inhalt (Methoden, Konventionen wie Zeilenzählung von oben/unten, Verhalten bei vollen Spalten, Feldgröße) wird **von allen Gruppen gemeinsam** erarbeitet: Jede Gruppe benennt eine Vertreterin/einen Vertreter; der Entwurf wird dem Dozenten zur Freigabe vorgelegt und von **einer** Person in das gemeinsame Verzeichnis committet.
- Nach der Freigabe sind die gemeinsamen Interfaces eingefroren. Änderungen nur nach Abstimmung aller Gruppen und Freigabe durch den Dozenten.

3.3 Verbindliche Regeln für die Implementierung

- Painter, Rules und Player einer Gruppe dürfen sich nur auf die gemeinsamen Interfaces (und die Framework-Basis-Interfaces) stützen – niemals auf konkrete Klassen der eigenen oder einer anderen Gruppe (kein Cast auf `A1_VierGewinntField` o. Ä.).
- `CanBePaintedBy` prüft auf `IC4_Painter`, `CanBeRuledBy` auf `IC4_Rules` – nicht auf eigene Klassentypen.
- Die Rules sind allein für die Gültigkeit verantwortlich: ungültige Züge (volle Spalte, Spalte außerhalb, falscher Spieler) werden abgewiesen; `CheckIfPLayerWon()` erkennt waagerecht, senkrecht und beide Diagonalen; `MovesPossible` erkennt ein volles Feld (Unentschieden).
- Der menschliche Player liefert bei ungültiger Eingabe `null` (vgl. Kommentar in `IGamePlayer.cs`). Die Umrechnung Mausposition → Spalte muss zur Geometrie passen, die *jeder* Painter verwendet – die Geometrie (Ursprung, Zellgröße bzw. Spaltenbreite) ist daher ebenfalls Teil der gemeinsamen Vereinbarung.
- Der Computer-Player liefert immer einen gültigen Zug (oder `null`, wenn keiner möglich ist) und soll eine erkennbare Strategie besitzen (mindestens: eigenen Gewinn nutzen, gegnerischen Gewinn blockieren).
- `Clone()` liefert ein unabhängiges Objekt; die Spielernummer aus `SetPlayerNumber` wird für alle Züge verwendet.

3.4 Nachweis der Kompatibilität

- Jede Gruppe testet ihre Komponenten mit den Komponenten **mindestens dreier anderer Gruppen** und dokumentiert das Ergebnis (Kompatibilitätsmatrix: wer mit wem, funktioniert ja/nein, gefundene Fehler) im Ordner `Scrum/`.

- Gefundene Fehler in fremden Komponenten werden der betreffenden Gruppe gemeldet (z. B. als GitHub-Issue) – nicht selbst in deren Dateien behoben.
- In der Abnahme wird eine zufällig gewählte Kombination aus Komponenten verschiedener Gruppen live gespielt.

4 Anforderung 2: Eigenes komplexes Spiel

4.1 Ziel

Jede Gruppe entwickelt mit Hilfe von KI-Agenten ein **eigenes, komplexes bzw. innovatives Spiel**. Eine Kompatibilität mit den Spielen anderer Gruppen ist **nicht** erforderlich. Weil Agenten einen großen Teil der Routinearbeit übernehmen, wird ein deutlich höherer Anspruch erwartet als bei Vier gewinnt – ein einfacher Klon eines Brettspiels reicht nicht aus.

Beispiele für angemessene Komplexität (Anregungen, keine Vorgabe):

- Echtzeitspiele über `IGameRules2.TickGameCall()` und `IPaintGame2.TickPaintGameField()` (Physik, Kollisionen, Animationen);
- Spiele mit nicht-trivialem Computergegner (Minimax/Alpha-Beta, Monte-Carlo-Suche, lernende Verfahren);
- Spiele mit mehreren Phasen, Levels, Ressourcen- oder Rundenlogik, Konfiguration beim Start über `StartedGameCall()`;
- kombinierte Maus- und Tastatursteuerung über `IClickSelection` / `IKeySelection`.

Das Spielkonzept ist dem Dozenten **vor Beginn der Implementierung** kurz vorzustellen (eine Seite als Plan im Ordner `Plans/`).

4.2 Harte Vorgabe: Das Spiel läuft im vorgegebenen Framework

- Das Spiel MUSS vollständig über die vorhandenen Framework-Interfaces funktionieren: Auswahl von Painter, Rules und Playern im Hauptfenster, Start über „Start Game“, Eingaben ausschließlich über die vom Framework gelieferten Klick-/Tastenereignisse, Zeichnen ausschließlich auf das übergebene Canvas.
- Kein eigenes Fenster, keine eigene Hauptschleife, kein Umgehen des Frameworks (z. B. eigene Event-Handler direkt am `MainWindow`).
- Gruppeneigene Interfaces (in `Interfaces/<Gruppe>/`) erweitern die Basis-Interfaces nach dem Muster von `X_Griesbauer`.
- Muss das Framework erweitert werden, gilt Abschnitt 2.4: **erst klären, dann ändern**. Der Agent darf das Framework nicht eigenständig anpassen.
- Die Interfaces des Frameworks sind mit *„DIESES INTERFACE NICHT ÄNDERN!“* markiert – das gilt ohne Ausnahme.

4.3 Qualität

- Sauberer objektorientierter Entwurf (Trennung Painter/Rules/Player, sinnvolle Klassen und Interfaces, keine „God-Class“).
- Das Team muss den gesamten Code – auch den agentisch erzeugten – **erklären können**. In der Abnahme werden einzelne Studierende zu beliebigen Stellen befragt.
- Das Projekt baut ohne Fehler; das Spiel stürzt bei üblicher Bedienung nicht ab.

4.4 Teamarbeit nach SCRUM

Die Entwicklung soll zeigen, dass die Gruppe Arbeit und Aufgaben gut koordiniert und verteilt. Dazu wird nach SCRUM gearbeitet und dies im Ordner `Classes/<Gruppe>/Scrum/` (alternativ über GitHub Issues/Projects, verlinkt aus `Scrum/README.md`) dokumentiert:

Element	Erwartung
Rollen	Product Owner und Scrum Master sind benannt; alle Mitglieder sind Entwickler.
Product Backlog	Anforderungen als User Stories mit Akzeptanzkriterien und Priorität.
Sprints	Sprints fester Länge (Empfehlung: 1–2 Wochen) mit Sprintziel und Sprint Backlog; jede Aufgabe hat eine verantwortliche Person.
Abstimmung	Kurzprotokolle der regelmäßigen Abstimmungen (Daily/Weekly): Was wurde erledigt, was ist geplant, was blockiert?
Review & Retrospektive	Je Sprint: Ergebnis (was ist fertig?) und Retrospektive (was verbessern wir?).
Nachvollziehbarkeit	Aufgabenverteilung im Backlog, Prompt-Dateien, Plänen und Commit-Historie passen zusammen. Jedes Mitglied hat einen substanziellen, sichtbaren Anteil.

Die Aufteilung soll so gewählt werden, dass parallel gearbeitet werden kann (z. B. Rules/Spiellogik, Painter/Darstellung, Computer-Player/KI, Test & Integration) und Konflikte in denselben Dateien vermieden werden.

5 Abgabe und Abnahme – Checkliste

✓	Kriterium
☐	Code liegt ausschließlich in `Classes/<Gruppe>/` und `Interfaces/<Gruppe>/` (plus Registrierung in `MainWindow.xaml.cs`).
☐	Keine nicht freigegebene Änderung am Framework.
☐	Jedes Teammitglied hat selbst committet und gepusht.
☐	Je Studierendem eine vollständige `Vorname_Nachname_Prompts.md`; alle Pläne in `Plans/`.
☐	Vier gewinnt: Painter, Rules, Human- und Computer-Player implementiert, nur gegen gemeinsame Interfaces programmiert.
☐	Vier gewinnt: Kompatibilität mit mindestens drei anderen Gruppen getestet und dokumentiert.
☐	Eigenes Spiel: Konzept vorab vorgestellt, komplex/innovativ, läuft vollständig im Framework.
☐	SCRUM-Dokumentation (Backlog, Sprints, Protokolle, Reviews/Retros) vorhanden und konsistent mit der Commit-Historie.
☐	Projekt baut fehlerfrei; das Team kann den gesamten Code erklären.

Termine für Konzeptvorstellung, Zwischenstände und Abnahme werden in der Vorlesung bekanntgegeben. Bei Unklarheiten gilt: vorher fragen.